

《星溯旅人》第一阶段：项目初始化 - 详细分步操作指南

部分内容由豆包生成

 **本指南目标：**从零开始搭建《星溯旅人》游戏项目的完整基础框架，包含4大模块、24个核心类，确保项目能正常编译运行。

预计耗时：2-3小时

前置条件：已安装Qt 6.10.1+、Qt Creator、CMake、Git

一、开发前准备

1.1 环境要求

项目	要求
Qt版本	Qt 6.10.1 或更高版本
Qt模块	Core、Gui、Widgets、Multimedia、Network、Svg、Xml
编译器	MinGW 64-bit 或 MSVC 2019+
CMake	3.20 或更高版本
C++标准	C++17
Git	任意最新版本

1.2 Qt Creator基础操作入门

如何创建新的C++类

- 在项目树中右键点击目标文件夹（如src/core/）
- 选择「新建」→「C++」→「C++ Class」
- 输入类名（如GameEngine），基类选择「无」或输入QObject

- 4. 点击「下一步」→「完成」，会自动生成.h和.cpp文件
- 5. CMakeLists.txt会自动添加新文件到项目中

如何构建和运行项目

- 左下角锤子图标：仅构建（编译）项目
- 左下角绿色三角图标：构建并运行项目
- 左下角虫子图标：调试模式运行
- 左下角电脑图标：切换Debug/Release构建模式
- 底部「编译输出」窗口：可查看编译错误和警告

常用快捷键

快捷键	功能
Ctrl+B	构建项目
Ctrl+R	运行项目
F5	开始调试
F9	设置/取消断点
F10	单步跳过
F11	单步进入
Ctrl+S	保存当前文件
Ctrl+Shift+S	保存所有文件
F2	跳转到符号定义
F4	切换头文件/源文件

二、步骤1：创建CMake项目

 **目标：**创建基础的Qt Widgets CMake项目，确保能正常编译运行

涉及文件：CMakeLists.txt、main.cpp

操作步骤

1. 打开Qt Creator，点击欢迎页面的「Create Project」或菜单栏「文件」→「新建文件或项目」
2. 在项目模板中选择「Application (Qt)」→「Qt Widgets Application」，点击「Choose...」
3. 填写项目名称：**StarRetrospectTraveler**，选择项目保存路径，点击「下一步」
4. 构建系统选择 **CMake**（默认），点击「下一步」
5. 基类选择 **QWidget**，类名填写MainWindow，保持「Generate form」勾选，点击「下一步」
6. 翻译文件可跳过，直接点击「下一步」
7. 构建套件选择：勾选MinGW 64-bit或MSVC（根据你安装的编译器），点击「下一步」
8. 版本控制系统选择「None」（后面单独配置Git），点击「下一步」
9. 检查项目设置无误后，点击「完成」创建项目
10. 项目创建完成后，点击左下角的绿色三角按钮（运行），测试能否正常编译运行
11. 如果弹出空白窗口，说明项目创建成功

验收标准

- ☐ 项目能成功编译
 - ☐ 能正常运行并显示空白窗口
 - ☐ CMakeLists.txt自动生成且配置正确
-

三、步骤2：配置编译选项



目标：配置Debug和Release两种编译模式，确保开发和发布都有合适的编译参数

操作步骤

1. 点击左侧「项目」按钮（或快捷键Ctrl+5）打开项目设置
2. 选择「构建与运行」→「Desktop Qt 6.10.1 MinGW 64-bit」
3. 在「构建」选项卡中，确认有Debug和Release两个构建配置
4. **Debug配置：**构建类型选Debug，勾选「启用调试信息」
5. **Release配置：**构建类型选Release，勾选「启用优化」
6. 点击左下角电脑图标，切换到Debug模式
7. 点击锤子图标（构建按钮），等待编译完成

8. 切换到Release模式，再次点击构建按钮，确认都能编译通过

修改CMakeLists.txt添加编译选项

打开根目录的CMakeLists.txt，添加以下编译配置：

CMakeLists.txt 编译配置部分

```
1  # C++标准设置
2  set(CMAKE_CXX_STANDARD 17)
3  set(CMAKE_CXX_STANDARD_REQUIRED ON)
4  set(CMAKE_CXX_EXTENSIONS OFF)
5
6  # 启用所有警告
7  if(MSVC)
8      add_compile_options(/W4)
9  else()
10     add_compile_options(-Wall -Wextra -Wpedantic)
11 endif()
12
13 # 编译选项配置
14 if(CMAKE_BUILD_TYPE STREQUAL "Debug")
15     # Debug模式：启用调试信息和AddressSanitizer
16     add_compile_options(-g -O0)
17     if(NOT MSVC)
18         add_compile_options(-fsanitize=address -fno-omit-frame-pointer)
19         add_link_options(-fsanitize=address)
20     endif()
21 elseif(CMAKE_BUILD_TYPE STREQUAL "Release")
22     # Release模式：启用O3优化
23     add_compile_options(-O3)
24 endif()
25
26 # MinGW-w64专用优化
27 if(MINGW)
28     add_compile_options(-static-libgcc -static-libstdc++)
29     add_link_options(-static-libgcc -static-libstdc++)
30     add_compile_options(-Wa,-mbig-obj)
31 endif()
```

验收标准

- ☐ Debug模式能正常编译
- ☐ Release模式能正常编译
- ☐ 警告信息正常显示

四、步骤3：建立目录结构

 **目标：**按模块化设计建立清晰的目录结构，便于后续开发和维护

最终结构：4大模块 + 资源 + 配置 + 文档

3.1 创建目录结构

1. 在Qt Creator左侧项目树中，右键点击项目根目录
2. 选择「Add New...」→「Directory」创建目录
3. 按以下结构依次创建所有目录：

完整项目目录结构

```
1  StarRetrospectTraveler/
2  |—— CMakeLists.txt          # CMake主配置文件
3  |—— main.cpp                # 程序入口
4  |
5  |—— src/                    # 【源代码目录】
6  |   |—— core/              # 核心引擎模块（8个类）
7  |   |—— game/              # 游戏逻辑模块（10个类）
8  |   |—— render/           # 渲染系统模块（5个类）
9  |   |—— ui/                # UI界面模块
10 |
11 |—— resources/              # 【资源文件目录】
12 |   |—— images/            # 图片资源
13 |   |—— audio/             # 音频资源
14 |   |—— fonts/             # 字体资源
15 |
16 |—— config/                 # 【配置文件目录】
17 |   |—— json/              # JSON数据配置
18 |   |—— qrc/               # Qt资源文件
19 |
20 |—— docs/                   # 文档目录
21 |   |—— design/            # 设计文档
```

3.2 移动初始文件

1. 将Qt Creator自动生成的main.cpp保留在根目录
2. 将mainwindow.h/.cpp/.ui移动到src/ui/目录（后续会替换为UIManager）

3. 更新CMakeLists.txt中的源文件路径

3.3 模块职责说明

模块	职责说明
src/core/	核心引擎模块，8个核心类：游戏主循环、场景管理、资源管理、输入管理、音频管理、配置管理、存档系统、插件管理
src/game/	游戏逻辑模块，10个类：三大核心系统 + 战斗/技能/AI/背包/任务 + 玩家和敌人实体
src/render/	渲染系统模块，5个类：基于Qt Graphics View实现2.5D渲染，相机、粒子、精灵动画、瓦片地图
src/ui/	UI界面模块：界面管理、Qt Designer可视化文件

验收标准

- ☐ 所有目录创建完成
- ☐ CMake能正确识别子目录
- ☐ 项目结构清晰，符合模块化设计

五、步骤4：配置Qt资源系统

 **目标：**配置Qt资源系统，管理图片、音频、字体等资源文件

操作步骤

- 在Qt Creator左侧项目树中，右键点击config/qrc目录
- 选择「Add New...」→「Qt」→「Qt Resource File」
- 文件名填写 **resources**，路径选择config/qrc/目录，点击「下一步」→「完成」
- 生成resources.qrc文件后，在资源编辑器中打开
- 点击「Add Prefix」添加前缀，按资源类型分类：
 - /images - 图片资源
 - /audio - 音频资源
 - /fonts - 字体资源

- /ui - UI素材
- 6. 点击「Add Files」，将resources目录下的资源文件添加到对应的前缀下
- 7. 在CMakeLists.txt中确认资源文件已被添加到项目中
- 8. 编译项目，确保资源能正确编译进可执行文件

resources.qrc示例

```
config/qrc/resources.qrc
1  <!DOCTYPE RCC>;
2  <RCC version="1.0">;
3      <qresource prefix="/images">;
4          <!-- 图片资源将在此添加 -->;
5      </qresource>;
6      <qresource prefix="/audio">;
7          <!-- 音频资源将在此添加 -->;
8      </qresource>;
9      <qresource prefix="/fonts">;
10         <!-- 字体资源将在此添加 -->;
11     </qresource>;
12     <qresource prefix="/ui">;
13         <!-- UI素材将在此添加 -->;
14     </qresource>;
15 </RCC>;
```

验收标准

- ☐ .qrc资源文件创建完成
 - ☐ 资源前缀按类型分类
 - ☐ 资源文件能成功编译进二进制

六、步骤5：配置Git版本控制



目标：初始化Git仓库，配置.gitignore，进行首次提交

操作步骤

1. 确保电脑上已安装Git，在Qt Creator中可以检测到Git（工具→选项→版本控制→Git）
2. 在Qt Creator菜单栏选择「工具」→「Git」→「创建仓库」

3. 选择项目根目录作为仓库路径，点击「确定」初始化Git仓库
4. 在项目根目录创建 **.gitignore** 文件
5. 在Qt Creator左侧「版本控制」面板中，查看所有文件状态
6. 选择所有需要提交的文件，右键选择「Add」添加到暂存区
7. 提交第一次提交：填写提交信息「Initial commit: 项目初始化」，点击「提交」
8. 配置Git用户信息：设置用户名和邮箱

.gitignore文件内容

```
.gitignore
1  # 构建目录
2  build-*/
3  build/
4  cmake-build-*/
5
6  # Qt Creator 用户配置
7  *.user
8  *.user.*
9  *.autosave
10
11 # Qt Creator 缓存
12 .qtc_clangd/
13 .qt/
14
15 # CMake 缓存
16 CMakeCache.txt
17 CMakeFiles/
18 cmake_install.cmake
19 install_manifest.txt
20
21 # 编译产物
22 *.o
23 *.obj
24 *.exe
25 *.dll
26 *.so
27 *.dylib
28 *.a
29 *.lib
30
31 # 可执行文件
32 StarRetrospectTraveler
33 StarRetrospectTraveler.exe
```



```
34
35 # 调试文件
36 *.pdb
37 *.ilk
38 *.exp
39
40 # IDE
41 .vscode/
42 .idea/
43 *.swp
44 *.swo
45 *~
46
47 # 日志文件
48 *.log
49 logs/
50
51 # 临时文件
52 tmp/
53 temp/
54 *.tmp
55
56 # 操作系统文件
57 .DS_Store
58 Thumbs.db
59
60 # 存档文件
61 saves/
62 *.sav
```

验收标准

- ☐ Git仓库初始化完成
- ☐ .gitignore配置正确
- ☐ 能正常提交代码

七、步骤6：创建核心引擎模块（core）



目标：创建核心引擎模块的8个基础类，搭建游戏引擎的核心框架

包含类：GameEngine、SceneManager、ResourceManager、InputManager、AudioManager、GameConfig、SaveSystem、PluginManager

6.1 创建core模块的CMakeLists.txt

在src/core/目录下创建CMakeLists.txt：

```
src/core/CMakeLists.txt

1  # 核心引擎模块
2  add_library(core STATIC
3      gameEngine.h
4      gameEngine.cpp
5      sceneManager.h
6      sceneManager.cpp
7      resourceManager.h
8      resourceManager.cpp
9      inputManager.h
10     inputManager.cpp
11     audioManager.h
12     audioManager.cpp
13     gameConfig.h
14     gameConfig.cpp
15     saveSystem.h
16     saveSystem.cpp
17     pluginManager.h
18     pluginManager.cpp
19 )
20
21 target_link_libraries(core PUBLIC
22     Qt6::Core
23     Qt6::Gui
24     Qt6::Widgets
25     Qt6::Multimedia
26     Qt6::Xml
27 )
28
29 target_include_directories(core PUBLIC
30     ${CMAKE_CURRENT_SOURCE_DIR}
31 )
```

6.2 创建GameEngine主类

文件：src/core/gameEngine.h 和 src/core/gameEngine.cpp

gameEngine.h 头文件

```
gameEngine.h

1  #ifndef GAMEENGINE_H
2  #define GAMEENGINE_H
3
4  #include <QObject>;
5  #include <QTimer>;
6  #include <QMainWindow>;
7  #include <memory>;
8
9  // 前向声明
10 class SceneManager;
11 class ResourceManager;
12 class InputManager;
13 class AudioManager;
14 class GameConfig;
15 class SaveSystem;
16 class PluginManager;
17
18 /**
19  * @class GameEngine
20  * @brief 游戏引擎核心类，负责游戏主循环和子系统管理
21  *
22  * GameEngine是整个游戏的核心管理器，负责：
23  * - 游戏主循环的控制（基于QTimer实现固定帧率）
24  * - 各子系统的初始化、更新和销毁
25  * - 游戏状态的管理
26  */
27 class GameEngine : public QObject
28 {
29     Q_OBJECT
30 public:
31     explicit GameEngine(QObject* parent = nullptr);
32     ~GameEngine() override;
33
34     bool initialize();          // 初始化游戏引擎
35     void start();              // 启动游戏主循环
36     void stop();              // 停止游戏主循环
37     bool isRunning() const;    // 游戏是否正在运行
38
39     // 获取各子系统管理器
40     SceneManager* getSceneManager() const;
41     ResourceManager* getResourceManager() const;
42     InputManager* getInputManager() const;
43     AudioManager* getAudioManager() const;
```

```

44     GameConfig* getGameConfig() const;
45     SaveSystem* getSaveSystem() const;
46     PluginManager* getPluginManager() const;
47     QMainWindow* getMainWindow() const;
48
49     signals:
50         void initialized(); // 游戏初始化完成信号
51         void started();     // 游戏启动信号
52         void stopped();     // 游戏停止信号
53
54     private slots:
55         void update(); // 游戏主循环更新函数
56
57     private:
58         bool initializeSubsystems(); // 初始化所有子系统
59         void shutdownSubsystems();   // 销毁所有子系统
60
61         QTimer* m_gameLoopTimer;     // 游戏主循环定时器
62         bool m_isRunning;             // 游戏运行状态
63         static constexpr int TARGET_FPS = 60; // 目标帧率
64         static constexpr int FRAME_INTERVAL_MS = 1000 / TARGET_FPS; // 帧间隔
65
66         QMainWindow* m_mainWindow;    // 主窗口
67
68         // 子系统管理器
69         std::unique_ptr<SceneManager> m_sceneManager;
70         std::unique_ptr<ResourceManager> m_resourceManager;
71         std::unique_ptr<InputManager> m_inputManager;
72         std::unique_ptr<AudioManager> m_audioManager;
73         std::unique_ptr<GameConfig> m_gameConfig;
74         std::unique_ptr<SaveSystem> m_saveSystem;
75         std::unique_ptr<PluginManager> m_pluginManager;
76     };
77
78 #endif // GAMEENGINE_H

```

gameEngine.cpp 实现文件

gameEngine.cpp

```

1  #include "gameEngine.h"
2  #include <QDebug>;
3  #include <QMainWindow>;
4
5  #include "sceneManager.h"
6  #include "resourceManager.h"

```

```
7  #include "inputManager.h"
8  #include "audioManager.h"
9  #include "gameConfig.h"
10 #include "saveSystem.h"
11 #include "pluginManager.h"
12
13 GameEngine::GameEngine(QObject* parent)
14     : QObject(parent)
15     , m_gameLoopTimer(new QTimer(this))
16     , m_isRunning(false)
17     , m_mainWindow(new QMainWindow())
18     , m_sceneManager(nullptr)
19     , m_resourceManager(nullptr)
20     , m_inputManager(nullptr)
21     , m_audioManager(nullptr)
22     , m_gameConfig(nullptr)
23     , m_saveSystem(nullptr)
24     , m_pluginManager(nullptr)
25 {
26     connect(m_gameLoopTimer, &QTimer::timeout, this, &GameEngine::update);
27 }
28
29 GameEngine::~GameEngine()
30 {
31     if (m_isRunning) { stop(); }
32     shutdownSubsystems();
33     if (m_mainWindow) { delete m_mainWindow; m_mainWindow = nullptr; }
34 }
35
36 bool GameEngine::initialize()
37 {
38     qDebug() << "[GameEngine] 初始化游戏引擎...";
39
40     // 设置主窗口
41     m_mainWindow->setWindowTitle("星溯旅人 - StarRetrospectTraveler");
42     m_mainWindow->resize(1280, 720);
43     m_mainWindow->setMinimumSize(800, 600);
44
45     // 初始化所有子系统
46     if (!initializeSubsystems()) {
47         qDebug() << "[GameEngine] 子系统初始化失败!";
48         return false;
49     }
50
51     emit initialized();
52     qDebug() << "[GameEngine] 游戏引擎初始化完成.";
53     return true;
```

```
54 }
55
56 void GameEngine::start()
57 {
58     if (m_isRunning) {
59         qWarning() << "[GameEngine] 游戏已经在运行中!";
60         return;
61     }
62     qDebug() << "[GameEngine] 启动游戏主循环, 目标帧率:" << TARGET_FPS << "FPS";
63     m_mainWindow->show();
64     m_gameLoopTimer->start(FRAME_INTERVAL_MS);
65     m_isRunning = true;
66     emit started();
67 }
68
69 void GameEngine::stop()
70 {
71     if (!m_isRunning) { return; }
72     qDebug() << "[GameEngine] 停止游戏主循环...";
73     m_gameLoopTimer->stop();
74     m_isRunning = false;
75     m_mainWindow->hide();
76     emit stopped();
77 }
78
79 bool GameEngine::isRunning() const { return m_isRunning; }
80
81 // 各子系统Getter实现 (略, 按格式实现)
82 SceneManager* GameEngine::getSceneManager() const { return
    m_sceneManager.get(); }
83 ResourceManager* GameEngine::getResourceManager() const { return
    m_resourceManager.get(); }
84 InputManager* GameEngine::getInputManager() const { return
    m_inputManager.get(); }
85 AudioManager* GameEngine::getAudioManager() const { return
    m_audioManager.get(); }
86 GameConfig* GameEngine::getGameConfig() const { return m_gameConfig.get(); }
87 SaveSystem* GameEngine::getSaveSystem() const { return m_saveSystem.get(); }
88 PluginManager* GameEngine::getPluginManager() const { return
    m_pluginManager.get(); }
89 QMainWindow* GameEngine::getMainWindow() const { return m_mainWindow; }
90
91 void GameEngine::update()
92 {
93     // TODO: 实现游戏逻辑更新
94     // 1. 处理输入
95     // 2. 更新游戏逻辑
```

```
96     // 3. 渲染场景
97     static int frameCount = 0;
98     frameCount++;
99     if (frameCount % TARGET_FPS == 0) {
100         qDebug() << "[GameEngine] 游戏运行中, 已运行" << frameCount / TARGET_FPS
101         << "秒";
102     }
103 }
104 bool GameEngine::initializeSubsystems()
105 {
106     qDebug() << "[GameEngine] 初始化子系统...";
107
108     // 初始化游戏配置
109     m_gameConfig = std::make_unique<GameConfig>();
110     if (!m_gameConfig->initialize()) { return false; }
111     qDebug() << "[GameEngine] ✓ 游戏配置初始化完成";
112
113     // 初始化资源管理器
114     m_resourceManager = std::make_unique<ResourceManager>();
115     if (!m_resourceManager->initialize()) { return false; }
116     qDebug() << "[GameEngine] ✓ 资源管理器初始化完成";
117
118     // 初始化输入管理器
119     m_inputManager = std::make_unique<InputManager>();
120     if (!m_inputManager->initialize()) { return false; }
121     qDebug() << "[GameEngine] ✓ 输入管理器初始化完成";
122
123     // 初始化音频管理器
124     m_audioManager = std::make_unique<AudioManager>();
125     if (!m_audioManager->initialize()) { return false; }
126     qDebug() << "[GameEngine] ✓ 音频管理器初始化完成";
127
128     // 初始化场景管理器
129     m_sceneManager = std::make_unique<SceneManager>();
130     if (!m_sceneManager->initialize()) { return false; }
131     qDebug() << "[GameEngine] ✓ 场景管理器初始化完成";
132
133     // 初始化存档系统
134     m_saveSystem = std::make_unique<SaveSystem>();
135     if (!m_saveSystem->initialize()) { return false; }
136     qDebug() << "[GameEngine] ✓ 存档系统初始化完成";
137
138     // 初始化插件管理器
139     m_pluginManager = std::make_unique<PluginManager>();
140     if (!m_pluginManager->initialize()) { return false; }
141     qDebug() << "[GameEngine] ✓ 插件管理器初始化完成";
```

```

142
143     qDebug() << "[GameEngine] 所有子系统初始化完成。";
144     return true;
145 }
146
147 void GameEngine::shutdownSubsystems()
148 {
149     qDebug() << "[GameEngine] 关闭子系统...";
150     // 按初始化逆序销毁
151     m_pluginManager.reset();
152     m_saveSystem.reset();
153     m_sceneManager.reset();
154     m_audioManager.reset();
155     m_inputManager.reset();
156     m_resourceManager.reset();
157     m_gameConfig.reset();
158     qDebug() << "[GameEngine] 所有子系统已关闭。";
159 }

```

6.3 创建SceneManager场景管理器

文件：src/core/sceneManager.h 和 src/core/sceneManager.cpp

sceneManager.h

```

1  #ifndef SCENEMANAGER_H
2  #define SCENEMANAGER_H
3
4  #include <QObject>;
5  #include <QGraphicsScene>;
6  #include <QGraphicsView>;
7  #include <QStack>;
8  #include <memory>;
9
10 class SceneManager : public QObject
11 {
12     Q_OBJECT
13 public:
14     explicit SceneManager(QObject* parent = nullptr);
15     ~SceneManager() override;
16
17     bool initialize();
18
19     // 场景切换
20     void switchToScene(const QString& sceneName);
21     void pushScene(const QString& sceneName);

```



```

22     void popScene();
23
24     // 获取当前场景
25     QGraphicsScene* getCurrentScene() const;
26     QGraphicsView* getView() const;
27
28 private:
29     QGraphicsView* m_view;
30     QGraphicsScene* m_currentScene;
31     QStack<QGraphicsScene*> m_sceneStack;
32     QMap<QString, QGraphicsScene*> m_sceneRegistry;
33 };
34
35 #endif // SCENEMANAGER_H

```

6.4 创建ResourceManager资源管理器

文件：src/core/resourceManager.h 和 src/core/resourceManager.cpp

resourceManager.h

```

1  #ifndef RESOURCEMANAGER_H
2  #define RESOURCEMANAGER_H
3
4  #include <QObject>;
5  #include <QPixmap>;
6  #include <QSoundEffect>;
7  #include <QFont>;
8  #include <QCache>;
9
10 class ResourceManager : public QObject
11 {
12     Q_OBJECT
13 public:
14     explicit ResourceManager(QObject* parent = nullptr);
15     ~ResourceManager() override;
16
17     bool initialize();
18
19     // 图片资源
20     QPixmap getPixmap(const QString& path);
21     bool preloadPixmap(const QString& path);
22
23     // 音效资源
24     QSoundEffect* getSoundEffect(const QString& path);
25

```

```

26     // 字体资源
27     QFont getFont(const QString& path, int pointSize = 12);
28
29     // 资源管理
30     void clearCache();
31     int getCacheSize() const;
32
33 private:
34     QCache<QString, QPixmap> m_pixmapCache;
35     QMap<QString, QSoundEffect*> m_soundCache;
36     QMap<QString, QFont> m_fontCache;
37
38     static constexpr int PIXMAP_CACHE_SIZE = 100;
39 };

```

6.5 创建InputManager输入管理器

文件：src/core/inputManager.h 和 src/core/inputManager.cpp

```

inputManager.h
1  #ifndef INPUTMANAGER_H
2  #define INPUTMANAGER_H
3
4  #include <QObject>
5  #include <QKeyEvent>
6  #include <QMouseEvent>
7  #include <QMap>
8  #include <QPoint>
9
10 class InputManager : public QObject
11 {
12     Q_OBJECT
13 public:
14     enum InputState {
15         STATE_RELEASED = 0,
16         STATE_PRESSED = 1,
17         STATE_HELD = 2
18     };
19
20     explicit InputManager(QObject* parent = nullptr);
21     ~InputManager() override;
22
23     bool initialize();
24
25     // 键盘状态查询

```

```

26     bool isKeyPressed(int key) const;
27     bool isKeyHeld(int key) const;
28     bool isKeyReleased(int key) const;
29     InputState getKeyState(int key) const;
30
31     // 鼠标状态查询
32     bool isMouseButtonPressed(Qt::MouseButton button) const;
33     QPoint getMousePosition() const;
34
35     // 输入映射
36     void setKeyMapping(const QString& action, int key);
37     bool isActionPressed(const QString& action) const;
38
39     // 每帧更新（用于状态转换）
40     void update();
41
42 protected:
43     bool eventFilter(QObject* obj, QEvent* event) override;
44
45 private:
46     QMap<int, InputState> m_keyStates;
47     QMap<int, InputState> m_mouseStates;
48     QMap<QString, int> m_keyMappings;
49     QPoint m_mousePosition;
50 };

```

6.6 创建AudioManager音频管理器

文件：src/core/audioManager.h 和 src/core/audioManager.cpp

```

audioManager.h
1  #ifndef AUDIOMANAGER_H
2  #define AUDIOMANAGER_H
3
4  #include <QObject>
5  #include <QMediaPlayer>
6  #include <QAudioOutput>
7  #include <QSoundEffect>
8
9  class AudioManager : public QObject
10 {
11     Q_OBJECT
12 public:
13     explicit AudioManager(QObject* parent = nullptr);
14     ~AudioManager() override;

```

```

15
16     bool initialize();
17
18     // 背景音乐
19     void playBGM(const QString& path);
20     void stopBGM();
21     void pauseBGM();
22     void resumeBGM();
23     void setBGMVolume(float volume);
24
25     // 音效
26     void playSound(const QString& path);
27     void setSoundVolume(float volume);
28
29     // 主音量
30     void setMasterVolume(float volume);
31     float getMasterVolume() const;
32
33 private:
34     QMediaPlayer* m_bgmPlayer;
35     QAudioOutput* m_bgmAudioOutput;
36     float m_masterVolume;
37     float m_bgmVolume;
38     float m_soundVolume;
39 };

```

6.7 创建GameConfig游戏配置

文件：src/core/gameConfig.h 和 src/core/gameConfig.cpp

```

gameConfig.h
1  #ifndef GAMECONFIG_H
2  #define GAMECONFIG_H
3
4  #include <QObject>
5  #include <QSettings>
6  #include <QVariant>
7
8  class GameConfig : public QObject
9  {
10     Q_OBJECT
11 public:
12     explicit GameConfig(QObject* parent = nullptr);
13     ~GameConfig() override;
14

```

```

15     bool initialize();
16
17     // 配置读写
18     QVariant getValue(const QString& key, const QVariant& defaultValue =
    QVariant()) const;
19     void setValue(const QString& key, const QVariant& value);
20
21     // 图形配置
22     int getWindowWidth() const;
23     int getWindowHeight() const;
24     bool isFullscreen() const;
25     bool isVSync() const;
26
27     // 音频配置
28     float getMasterVolume() const;
29     float getBGMVolume() const;
30     float getSoundVolume() const;
31
32     // 游戏配置
33     QString getLanguage() const;
34     bool isAutoSave() const;
35
36     // 重置配置
37     void resetToDefaults();
38     void sync();
39
40 signals:
41     void configChanged(const QString& key, const QVariant& value);
42
43 private:
44     QSettings* m_settings;
45     void setDefaultValues();
46 };

```

6.8 创建SaveSystem存档系统

文件：src/core/saveSystem.h 和 src/core/saveSystem.cpp

saveSystem.h

```

1  #ifndef SAVESYSTEM_H
2  #define SAVESYSTEM_H
3
4  #include <QObject>;
5  #include <QJsonObject>;
6  #include <QJsonDocument>;

```

```

7  #include <QStringList>;
8
9  class SaveSystem : public QObject
10 {
11     Q_OBJECT
12 public:
13     struct SaveInfo {
14         int slot;
15         QString name;
16         QString timestamp;
17         QString playTime;
18         QString location;
19     };
20
21     explicit SaveSystem(QObject* parent = nullptr);
22     ~SaveSystem() override;
23
24     bool initialize();
25
26     // 存档操作
27     bool saveGame(int slot, const QString& name, const QJsonObject& data);
28     bool loadGame(int slot, QJsonObject& data);
29     bool deleteSave(int slot);
30
31     // 存档信息
32     QList<SaveInfo> getSaveList() const;
33     bool hasSave(int slot) const;
34     QString getSavePath(int slot) const;
35
36     // 自动存档
37     void setAutoSaveEnabled(bool enabled);
38     bool isAutoSaveEnabled() const;
39
40 private:
41     QString m_saveDir;
42     bool m_autoSaveEnabled;
43
44     QString getSaveFileName(int slot) const;
45     bool ensureSaveDirectory();
46 };

```

6.9 创建PluginManager插件管理器

文件：src/core/pluginManager.h 和 src/core/pluginManager.cpp

pluginManager.h

```
1  #ifndef PLUGINMANAGER_H
2  #define PLUGINMANAGER_H
3
4  #include <QObject>;
5  #include <QPluginLoader>;
6  #include <QList>;
7  #include <QMap>;
8  #include <QJsonObject>;
9
10 class PluginManager : public QObject
11 {
12     Q_OBJECT
13 public:
14     struct PluginInfo {
15         QString id;
16         QString name;
17         QString version;
18         QString author;
19         QString description;
20         bool isLoading;
21     };
22
23     explicit PluginManager(QObject* parent = nullptr);
24     ~PluginManager() override;
25
26     bool initialize();
27
28     // 插件管理
29     bool loadPlugin(const QString& path);
30     bool unloadPlugin(const QString& id);
31     bool isPluginLoaded(const QString& id) const;
32
33     // 插件信息
34     QList<PluginInfo> getPluginList() const;
35     QObject* getPluginInstance(const QString& id) const;
36
37     // 插件目录
38     void setPluginDirectory(const QString& path);
39     QString getPluginDirectory() const;
40     void scanPlugins();
41
42 private:
43     QString m_pluginDirectory;
44     QMap<QString, QPluginLoader*> m_plugins;
45     QMap<QString, PluginInfo> m_pluginInfo;
46 };
```

验收标准

- ☐ core模块CMakeLists.txt配置正确
- ☐ 8个类的.h和.cpp文件全部创建
- ☐ 每个类都有initialize()方法
- ☐ GameEngine能正确初始化所有子系统
- ☐ 代码能正常编译通过

八、步骤7：创建游戏逻辑模块（game）



目标：创建游戏逻辑模块的10个基础类，搭建游戏玩法框架

包含类：Player、Enemy、TimeSystem、ResonanceSystem、EcosystemSystem、CombatSystem、SkillSystem、AISystem、InventorySystem、QuestSystem

7.1 创建game模块的CMakeLists.txt

src/game/CMakeLists.txt

```
1  # 游戏逻辑模块
2  add_library(game STATIC
3      player.h
4      player.cpp
5      enemy.h
6      enemy.cpp
7      timeSystem.h
8      timeSystem.cpp
9      resonanceSystem.h
10     resonanceSystem.cpp
11     ecosystemSystem.h
12     ecosystemSystem.cpp
13     combatSystem.h
14     combatSystem.cpp
15     skillSystem.h
16     skillSystem.cpp
17     aiSystem.h
18     aiSystem.cpp
19     inventorySystem.h
20     inventorySystem.cpp
21     questSystem.h
22     questSystem.cpp
23 )
```



```

24
25 target_link_libraries(game PUBLIC
26     Qt6::Core
27     Qt6::Gui
28     Qt6::Widgets
29     core
30 )
31
32 target_include_directories(game PUBLIC
33     ${CMAKE_CURRENT_SOURCE_DIR}
34 )

```

7.2 创建Player玩家类

```

player.h
1  #ifndef PLAYER_H
2  #define PLAYER_H
3
4  #include <QObject>;
5  #include <QPointF>;
6  #include <QString>;
7
8  class Player : public QObject
9  {
10     Q_OBJECT
11     public:
12         enum PlayerState {
13             STATE_IDLE,
14             STATE_WALKING,
15             STATE_RUNNING,
16             STATE_ATTACKING,
17             STATE_HURT,
18             STATE_DEAD
19         };
20
21         explicit Player(QObject* parent = nullptr);
22         ~Player() override;
23
24         // 属性
25         int getMaxHP() const;
26         int getCurrentHP() const;
27         int getAttack() const;
28         int getDefense() const;
29         float getMoveSpeed() const;
30         int getLevel() const;

```

```
31     int getExp() const;
32     int getExpToNextLevel() const;
33
34     // 位置
35     QPointF getPosition() const;
36     void setPosition(const QPointF& pos);
37     void move(float dx, float dy);
38
39     // 状态
40     PlayerState getState() const;
41     void setState(PlayerState state);
42
43     // 战斗
44     void takeDamage(int damage);
45     void heal(int amount);
46     void gainExp(int exp);
47     bool isAlive() const;
48
49     // 更新
50     void update(float deltaTime);
51
52 signals:
53     void hpChanged(int current, int max);
54     void expChanged(int current, int max);
55     void levelUp(int newLevel);
56     void playerDied();
57     void stateChanged(PlayerState newState);
58
59 private:
60     // 基础属性
61     int m_maxHP;
62     int m_currentHP;
63     int m_attack;
64     int m_defense;
65     float m_moveSpeed;
66     int m_level;
67     int m_exp;
68
69     // 状态
70     PlayerState m_state;
71     QPointF m_position;
72
73     // 方法
74     void levelUp();
75     void calculateStats();
76 };
77
```

7.3 创建Enemy敌人基类

```

enemy.h

1  #ifndef ENEMY_H
2  #define ENEMY_H
3
4  #include <QObject>
5  #include <QPointF>
6  #include <QString>
7
8  class Enemy : public QObject
9  {
10     Q_OBJECT
11 public:
12     enum EnemyState {
13         STATE_IDLE,
14         STATE_PATROL,
15         STATE_CHASE,
16         STATE_ATTACK,
17         STATE_HURT,
18         STATE_DEAD
19     };
20
21     enum EnemyType {
22         TYPE_NORMAL,
23         TYPE_ELITE,
24         TYPE_BOSS
25     };
26
27     explicit Enemy(QObject* parent = nullptr);
28     ~Enemy() override;
29
30     // 初始化
31     void initialize(const QString& enemyId);
32
33     // 属性
34     int getMaxHP() const;
35     int getCurrentHP() const;
36     int getAttack() const;
37     int getDefense() const;
38     float getMoveSpeed() const;
39     int getExpReward() const;
40     EnemyType getType() const;

```

```
41
42     // AI
43     EnemyState getState() const;
44     void setState(EnemyState state);
45     void updateAI(float deltaTime, const QPointF& playerPos);
46
47     // 位置
48     QPointF getPosition() const;
49     void setPosition(const QPointF& pos);
50
51     // 战斗
52     void takeDamage(int damage);
53     bool isAlive() const;
54
55     signals:
56         void enemyDied();
57         void stateChanged(EnemyState newState);
58
59     private:
60         QString m_enemyId;
61         EnemyType m_type;
62         EnemyState m_state;
63
64         int m_maxHP;
65         int m_currentHP;
66         int m_attack;
67         int m_defense;
68         float m_moveSpeed;
69         int m_expReward;
70
71         float m_detectRange;
72         float m_attackRange;
73         float m_attackCooldown;
74         float m_currentCooldown;
75
76         QPointF m_position;
77         QPointF m_patrolTarget;
78
79         void loadFromConfig(const QString& enemyId);
80         void patrolBehavior(float deltaTime);
81         void chaseBehavior(float deltaTime, const QPointF& playerPos);
82         void attackBehavior(float deltaTime, const QPointF& playerPos);
83     };
84
85     #endif // ENEMY_H
```

7.4 创建三大核心系统

TimeSystem 时间褶皱系统

```
timeSystem.h

1  #ifndef TIMESYSTEM_H
2  #define TIMESYSTEM_H
3
4  #include <QObject>
5
6  class TimeSystem : public QObject
7  {
8      Q_OBJECT
9  public:
10     enum TimeState {
11         TIME_PRESENT,
12         TIME_PAST,
13         TIME_TRANSITIONING
14     };
15
16     explicit TimeSystem(QObject* parent = nullptr);
17     ~TimeSystem() override;
18
19     bool initialize();
20
21     // 时间褶皱
22     bool triggerWarp();
23     TimeState getCurrentTime() const;
24     bool isTransitioning() const;
25
26     // 能量系统
27     float getCurrentEnergy() const;
28     float getMaxEnergy() const;
29     float getEnergyRegenRate() const;
30     void consumeEnergy(float amount);
31     void regenerateEnergy(float deltaTime);
32     bool hasEnoughEnergy(float amount) const;
33
34     // 时间效果
35     float getTimeScale() const;
36     void setTimeScale(float scale);
37     void slowMotion(float duration, float scale);
38
39     void update(float deltaTime);
40
41     signals:
```

```

42     void timeStateChanged(TimeState newState);
43     void energyChanged(float current, float max);
44     void warpTriggered();
45
46 private:
47     TimeState m_currentTime;
48     float m_currentEnergy;
49     float m_maxEnergy;
50     float m_energyRegenRate;
51     float m_timeScale;
52     float m_slowMotionTimer;
53     float m_slowMotionScale;
54 };
55
56 #endif // TIMESYSTEM_H

```

ResonanceSystem 星核共鸣系统

```

resonanceSystem.h

1  #ifndef RESONANCESYSTEM_H
2  #define RESONANCESYSTEM_H
3
4  #include <QObject>;
5  #include <QMap>;
6  #include <QString>;
7
8  class ResonanceSystem : public QObject
9  {
10     Q_OBJECT
11 public:
12     enum ResonanceType {
13         RESONANCE_NONE,
14         RESONANCE_LIGHT,
15         RESONANCE_DARK,
16         RESONANCE_EARTH,
17         RESONANCE_WATER,
18         RESONANCE_FIRE,
19         RESONANCE_WIND
20     };
21
22     explicit ResonanceSystem(QObject* parent = nullptr);
23     ~ResonanceSystem() override;
24
25     bool initialize();
26

```

```

27     // 共鸣等级
28     int getResonanceLevel(ResonanceType type) const;
29     void increaseResonance(ResonanceType type, int amount);
30     int getTotalResonanceLevel() const;
31
32     // 共鸣效果
33     float getResonanceBonus(ResonanceType type) const;
34     bool isResonanceActive(ResonanceType type) const;
35
36     // 星核收集
37     int getStarCoresCollected() const;
38     void collectStarCore(const QString& coreId);
39     bool hasStarCore(const QString& coreId) const;
40
41     // 共鸣技能
42     QStringList getAvailableSkills() const;
43     bool canUseSkill(const QString& skillId) const;
44
45     signals:
46         void resonanceLevelChanged(ResonanceType type, int newLevel);
47         void starCoreCollected(const QString& coreId);
48
49     private:
50         QMap<ResonanceType, int>& m_resonanceLevels;
51         QMap<QString, bool>& m_collectedCores;
52         int m_totalCores;
53
54         void calculateResonanceBonuses();
55     };
56
57 #endif // RESONANCESYSTEM_H

```

EcosystemSystem 生态共生系统

```

ecosystemSystem.h
1  #ifndef ECOSYSTEMSYSTEM_H
2  #define ECOSYSTEMSYSTEM_H
3
4  #include <QObject>
5  #include <QList>
6  #include <QString>
7
8  class EcosystemSystem : public QObject
9  {
10     Q_OBJECT

```

```
11 public:
12     struct Creature {
13         QString id;
14         QString name;
15         QString type;
16         int friendshipLevel;
17         bool isSummoned;
18         QString ability;
19     };
20
21     explicit EcosystemSystem(QObject* parent = nullptr);
22     ~EcosystemSystem() override;
23
24     bool initialize();
25
26     // 共生生物
27     QList<Creature> getCreatures() const;
28     Creature getCreature(const QString& id) const;
29     bool hasCreature(const QString& id) const;
30     void addCreature(const Creature& creature);
31
32     // 友好度
33     int getFriendshipLevel(const QString& creatureId) const;
34     void increaseFriendship(const QString& creatureId, int amount);
35
36     // 召唤
37     bool summonCreature(const QString& creatureId);
38     void dismissCreature(const QString& creatureId);
39     QList<Creature> getSummonedCreatures() const;
40
41     // 生态平衡
42     float getEcosystemBalance() const;
43     void updateBalance(float deltaTime);
44
45     // 能力
46     QStringList getActiveAbilities() const;
47
48 signals:
49     void creatureAdded(const QString& creatureId);
50     void friendshipChanged(const QString& creatureId, int level);
51     void creatureSummoned(const QString& creatureId);
52     void balanceChanged(float balance);
53
54 private:
55     QList<Creature> m_creatures;
56     QList<QString> m_summonedIds;
57     float m_ecosystemBalance;
```



```
58     int m_maxSummoned;
59 };
60
61 #endif // ECOSYSTEMSYSTEM_H
```

7.5 创建战斗、技能、AI系统

CombatSystem 战斗系统

```
combatSystem.h

1  #ifndef COMBATSYSTEM_H
2  #define COMBATSYSTEM_H
3
4  #include <QObject>
5  #include <QList>
6
7  class Player;
8  class Enemy;
9
10 class CombatSystem : public QObject
11 {
12     Q_OBJECT
13 public:
14     enum CombatState {
15         STATE_NONE,
16         STATE_IN_COMBAT,
17         STATE_VICTORY,
18         STATE_DEFEAT
19     };
20
21     explicit CombatSystem(QObject* parent = nullptr);
22     ~CombatSystem() override;
23
24     bool initialize();
25
26     // 战斗状态
27     CombatState getCombatState() const;
28     void startCombat();
29     void endCombat(bool victory);
30
31     // 伤害计算
32     int calculateDamage(int attackerAttack, int defenderDefense, float
multiplier = 1.0f);
33     bool isCriticalHit(float critChance) const;
34
```

```

35     // 战斗更新
36     void update(float deltaTime);
37
38     // 敌人管理
39     void addEnemy(Enemy* enemy);
40     void removeEnemy(Enemy* enemy);
41     QList<Enemy*> getEnemies() const;
42     int getAliveEnemyCount() const;
43
44     void setPlayer(Player* player);
45
46     signals:
47         void combatStarted();
48         void combatEnded(bool victory);
49         void damageDealt(int damage, bool isCritical);
50         void enemyKilled(Enemy* enemy);
51
52     private:
53         CombatState m_combatState;
54         Player* m_player;
55         QList<Enemy*> m_enemies;
56         float m_combatTimer;
57
58         void checkCombatEnd();
59 };
60
61 #endif // COMBATSYSTEM_H

```

SkillSystem 技能系统

skillSystem.h

```

1  #ifndef SKILLSYSTEM_H
2  #define SKILLSYSTEM_H
3
4  #include <QObject>
5  #include <QMap>
6  #include <QString>
7
8  class SkillSystem : public QObject
9  {
10     Q_OBJECT
11     public:
12         struct Skill {
13             QString id;
14             QString name;

```

```

15         QString description;
16         int damage;
17         float cooldown;
18         float currentCooldown;
19         int energyCost;
20         QString type;
21         bool isUnlocked;
22     };
23
24     explicit SkillSystem(QObject* parent = nullptr);
25     ~SkillSystem() override;
26
27     bool initialize();
28
29     // 技能管理
30     bool hasSkill(const QString& skillId) const;
31     void unlockSkill(const QString& skillId);
32     Skill getSkill(const QString& skillId) const;
33     QList<Skill> getAllSkills() const;
34     QList<Skill> getUnlockedSkills() const;
35
36     // 技能使用
37     bool canUseSkill(const QString& skillId) const;
38     bool useSkill(const QString& skillId);
39     float getCooldownProgress(const QString& skillId) const;
40
41     // 更新
42     void update(float deltaTime);
43
44     signals:
45         void skillUnlocked(const QString& skillId);
46         void skillUsed(const QString& skillId);
47         void skillCooldownFinished(const QString& skillId);
48
49     private:
50         QMap<QString, Skill> m_skills;
51
52         void initializeSkills();
53     };
54
55 #endif // SKILLSYSTEM_H

```

AISystem AI系统

aiSystem.h

```
1  #ifndef AISYSTEM_H
2  #define AISYSTEM_H
3
4  #include <QObject>;
5  #include <QList>;
6
7  class Enemy;
8  class Player;
9
10 class AISystem : public QObject
11 {
12     Q_OBJECT
13 public:
14     enum BehaviorType {
15         BEHAVIOR_PASSIVE,
16         BEHAVIOR_AGGRESSIVE,
17         BEHAVIOR_DEFENSIVE,
18         BEHAVIOR_SUPPORT
19     };
20
21     explicit AISystem(QObject* parent = nullptr);
22     ~AISystem() override;
23
24     bool initialize();
25
26     // AI更新
27     void update(float deltaTime);
28
29     // 敌人管理
30     void registerEnemy(Enemy* enemy);
31     void unregisterEnemy(Enemy* enemy);
32
33     // 行为配置
34     void setBehaviorType(Enemy* enemy, BehaviorType type);
35     BehaviorType getBehaviorType(Enemy* enemy) const;
36
37     // 感知系统
38     float getDetectionRange(Enemy* enemy) const;
39     float getAttackRange(Enemy* enemy) const;
40
41     void setPlayer(Player* player);
42
43 private:
44     QList<Enemy*> m_enemies;
45     QMap<Enemy*, BehaviorType> m_behaviorTypes;
46     Player* m_player;
47
```

```

48     void updateEnemyAI(Enemy* enemy, float deltaTime);
49     bool canDetectPlayer(Enemy* enemy) const;
50     float getDistanceToPlayer(Enemy* enemy) const;
51 };
52
53 #endif // AISYSTEM_H

```

7.6 创建背包和任务系统

InventorySystem 背包系统

```

inventorySystem.h

1  #ifndef INVENTORYSYSTEM_H
2  #define INVENTORYSYSTEM_H
3
4  #include <QObject>;
5  #include <QList>;
6  #include <QMap>;
7  #include <QString>;
8
9  class InventorySystem : public QObject
10 {
11     Q_OBJECT
12 public:
13     struct Item {
14         QString id;
15         QString name;
16         QString description;
17         QString type;
18         int quantity;
19         int maxStack;
20         bool isStackable;
21         int value;
22         QString icon;
23     };
24
25     enum InventorySlot {
26         SLOT_WEAPON,
27         SLOT_ARMOR,
28         SLOT_ACCESSORY1,
29         SLOT_ACCESSORY2,
30         SLOT_COUNT
31     };
32
33     explicit InventorySystem(QObject* parent = nullptr);

```

```

34     ~InventorySystem() override;
35
36     bool initialize();
37
38     // 物品管理
39     bool addItem(const QString& itemId, int quantity = 1);
40     bool removeItem(const QString& itemId, int quantity = 1);
41     int getItemCount(const QString& itemId) const;
42     bool hasItem(const QString& itemId, int quantity = 1) const;
43     Item getItem(const QString& itemId) const;
44
45     // 背包容量
46     int getMaxSlots() const;
47     int getUsedSlots() const;
48     QList<Item> getItems() const;
49
50     // 装备系统
51     bool equipItem(const QString& itemId, InventorySlot slot);
52     bool unequipItem(InventorySlot slot);
53     Item getEquippedItem(InventorySlot slot) const;
54     bool isEquipped(const QString& itemId) const;
55
56     // 金币
57     int getGold() const;
58     void addGold(int amount);
59     bool spendGold(int amount);
60
61     signals:
62         void itemAdded(const QString& itemId, int quantity);
63         void itemRemoved(const QString& itemId, int quantity);
64         void itemEquipped(const QString& itemId, InventorySlot slot);
65         void goldChanged(int amount);
66
67     private:
68         QMap<QString, Item> m_items;
69         QMap<InventorySlot, QString> m_equippedItems;
70         int m_gold;
71         int m_maxSlots;
72
73         void loadItemData(const QString& itemId, Item& item);
74 };
75
76 #endif // INVENTORYSYSTEM_H

```

QuestSystem 任务系统

```

1 #ifndef QUESTSYSTEM_H
1 questSystem.h
2 #define QUESTSYSTEM_H
3
4 #include <QObject>;
5 #include <QList>;
6 #include <QMap>;
7 #include <QString>;
8
9 class QuestSystem : public QObject
10 {
11     Q_OBJECT
12 public:
13     enum QuestState {
14         QUEST_UNAVAILABLE,
15         QUEST_AVAILABLE,
16         QUEST_IN_PROGRESS,
17         QUEST_COMPLETED,
18         QUEST_FAILED
19     };
20
21     struct QuestObjective {
22         QString id;
23         QString description;
24         QString type;
25         int targetCount;
26         int currentCount;
27         bool isCompleted;
28     };
29
30     struct Quest {
31         QString id;
32         QString name;
33         QString description;
34         QuestState state;
35         QList<QuestObjective> objectives;
36         int expReward;
37         int goldReward;
38         QStringList itemRewards;
39         QString questGiver;
40     };
41
42     explicit QuestSystem(QObject* parent = nullptr);
43     ~QuestSystem() override;
44
45     bool initialize();
46
47     // 任务管理

```

```

48     bool acceptQuest(const QString& questId);
49     bool completeQuest(const QString& questId);
50     bool abandonQuest(const QString& questId);
51     QuestState getQuestState(const QString& questId) const;
52     Quest getQuest(const QString& questId) const;
53
54     // 任务列表
55     QList<Quest> getActiveQuests() const;
56     QList<Quest> getCompletedQuests() const;
57     QList<Quest> getAvailableQuests() const;
58
59     // 任务进度
60     void updateObjective(const QString& questId, const QString& objectiveId,
61         int progress = 1);
62     bool isQuestComplete(const QString& questId) const;
63
64     // 任务追踪
65     QString getTrackedQuest() const;
66     void setTrackedQuest(const QString& questId);
67
68 signals:
69     void questAccepted(const QString& questId);
70     void questCompleted(const QString& questId);
71     void questObjectiveUpdated(const QString& questId, const QString&
72         objectiveId);
73     void questFailed(const QString& questId);
74
75 private:
76     QMap<QString, Quest> m_quests;
77     QString m_trackedQuestId;
78
79     void initializeQuests();
80     void checkQuestCompletion(const QString& questId);
81     void giveRewards(const Quest& quest);
82 };
83
84 #endif // QUESTSYSTEM_H

```

验收标准

- ☐ game模块CMakeLists.txt配置正确
- ☐ 10个类的.h和.cpp文件全部创建
- ☐ Player和Enemy类有完整的属性和状态系统
- ☐ 三大核心系统有基础框架

☐ 战斗、技能、AI、背包、任务系统有基础接口

☐ 代码能正常编译通过

九、步骤8：创建渲染系统模块（render）



目标：创建渲染系统模块的5个基础类，搭建2.5D渲染框架

包含类：RenderManager、CameraSystem、ParticleSystem、SpriteAnimation、TileMap

8.1 创建render模块的CMakeLists.txt

src/render/CMakeLists.txt

```
1  # 渲染系统模块
2  add_library(render STATIC
3      renderManager.h
4      renderManager.cpp
5      cameraSystem.h
6      cameraSystem.cpp
7      particleSystem.h
8      particleSystem.cpp
9      spriteAnimation.h
10     spriteAnimation.cpp
11     tileMap.h
12     tileMap.cpp
13 )
14
15 target_link_libraries(render PUBLIC
16     Qt6::Core
17     Qt6::Gui
18     Qt6::Widgets
19     Qt6::Svg
20     core
21 )
22
23 target_include_directories(render PUBLIC
24     ${CMAKE_CURRENT_SOURCE_DIR}
25 )
```

8.2 创建RenderManager渲染管理器

```
1  #ifndef RENDERMANAGER_H
2  #define RENDERMANAGER_H
3
4  #include <QObject>;
5  #include <QGraphicsScene>;
6  #include <QGraphicsView>;
7  #include <QList>;
8
9  class CameraSystem;
10 class ParticleSystem;
11
12 class RenderManager : public QObject
13 {
14     Q_OBJECT
15 public:
16     explicit RenderManager(QObject* parent = nullptr);
17     ~RenderManager() override;
18
19     bool initialize();
20
21     // 场景管理
22     QGraphicsScene* getScene() const;
23     QGraphicsView* getView() const;
24     void setScene(QGraphicsScene* scene);
25
26     // 相机系统
27     CameraSystem* getCamera() const;
28
29     // 粒子系统
30     ParticleSystem* getParticleSystem() const;
31
32     // 渲染设置
33     void setClearColor(const QColor& color);
34     QColor getClearColor() const;
35
36     // 渲染更新
37     void update(float deltaTime);
38     void render();
39
40     // 视口
41     QRectF getViewport() const;
42     void resizeViewport(int width, int height);
43
44 private:
45     QGraphicsScene* m_scene;
46     QGraphicsView* m_view;
47     CameraSystem* m_camera;
```

```
48     ParticleSystem* m_particleSystem;
49     QColor m_clearColor;
50 };
51
52 #endif // RENDERMANAGER_H
```

8.3 创建CameraSystem相机系统

```
cameraSystem.h

1  #ifndef CAMERASYSTEM_H
2  #define CAMERASYSTEM_H
3
4  #include <QObject>;
5  #include <QPointF>;
6  #include <QRectF>;
7  #include <QSizeF>;
8
9  class CameraSystem : public QObject
10 {
11     Q_OBJECT
12 public:
13     explicit CameraSystem(QObject* parent = nullptr);
14     ~CameraSystem() override;
15
16     bool initialize();
17
18     // 位置
19     QPointF getPosition() const;
20     void setPosition(const QPointF& pos);
21     void move(const QPointF& delta);
22
23     // 跟随目标
24     void setFollowTarget(const QPointF* target);
25     void setSmoothFollowEnabled(bool enabled);
26     void setFollowSpeed(float speed);
27
28     // 缩放
29     float getZoom() const;
30     void setZoom(float zoom);
31     void zoomIn(float amount);
32     void zoomOut(float amount);
33
34     // 边界限制
35     void setBounds(const QRectF& bounds);
36     QRectF getBounds() const;
```

```

37     void setBoundsEnabled(bool enabled);
38
39     // 屏幕震动
40     void shake(float intensity, float duration);
41     bool isShaking() const;
42
43     // 视口
44     void setViewportSize(const QSizeF& size);
45     QSizeF getViewportSize() const;
46     QRectF getViewRect() const;
47
48     // 世界坐标与屏幕坐标转换
49     QPointF screenToWorld(const QPointF& screenPos) const;
50     QPointF worldToScreen(const QPointF& worldPos) const;
51
52     void update(float deltaTime);
53
54 private:
55     QPointF m_position;
56     float m_zoom;
57     QSizeF m_viewportSize;
58     QRectF m_bounds;
59     bool m_boundsEnabled;
60
61     const QPointF* m_followTarget;
62     bool m_smoothFollowEnabled;
63     float m_followSpeed;
64
65     float m_shakeIntensity;
66     float m_shakeDuration;
67     float m_shakeTimer;
68     QPointF m_shakeOffset;
69
70     void clampPositionToBounds();
71     void updateShake(float deltaTime);
72     void updateFollow(float deltaTime);
73 };
74
75 #endif // CAMERASYSTEM_H

```

8.4 创建ParticleSystem粒子系统

particleSystem.h

```

1  #ifndef PARTICLESYSTEM_H
2  #define PARTICLESYSTEM_H

```

```

3
4  #include <QObject>;
5  #include <QList>;
6  #include <QPointF>;
7  #include <QColor>;
8  #include <QPixmap>;
9
10 class ParticleSystem : public QObject
11 {
12     Q_OBJECT
13 public:
14     struct Particle {
15         QPointF position;
16         QPointF velocity;
17         QColor color;
18         float size;
19         float life;
20         float maxLife;
21         float rotation;
22         float rotationSpeed;
23         bool isActive;
24     };
25
26     struct Emitter {
27         QString id;
28         QPointF position;
29         int maxParticles;
30         float emissionRate;
31         float particleLife;
32         float particleSize;
33         QColor startColor;
34         QColor endColor;
35         QPointF direction;
36         float spread;
37         float speedMin;
38         float speedMax;
39         bool isActive;
40     };
41
42     explicit ParticleSystem(QObject* parent = nullptr);
43     ~ParticleSystem() override;
44
45     bool initialize();
46
47     // 发射器管理
48     void addEmitter(const Emitter& emitter);
49     void removeEmitter(const QString& id);

```

```

50     void startEmitter(const QString& id);
51     void stopEmitter(const QString& id);
52     bool isEmitterActive(const QString& id) const;
53
54     // 粒子操作
55     void emitParticles(const QPointF& position, int count, const QString&
emitterId = "default");
56     void clearAllParticles();
57
58     // 更新
59     void update(float deltaTime);
60
61     // 性能
62     int getActiveParticleCount() const;
63     void setMaxParticles(int max);
64
65 private:
66     QList<Particle> m_particles;
67     QMap<QString, Emitter> m_emitters;
68     int m_maxParticles;
69
70     void spawnParticle(const Emitter& emitter);
71     void updateParticle(Particle& particle, float deltaTime);
72 };
73
74 #endif // PARTICLESYSTEM_H

```

8.5 创建SpriteAnimation精灵动画

spriteAnimation.h

```

1  #ifndef SPRITEANIMATION_H
2  #define SPRITEANIMATION_H
3
4  #include <QObject>
5  #include <QPixmap>
6  #include <QList>
7  #include <QString>
8  #include <QMap>
9
10 class SpriteAnimation : public QObject
11 {
12     Q_OBJECT
13 public:
14     enum PlayMode {
15         PLAY_ONCE,

```

```

16         PLAY_LOOP,
17         PLAY_PINGPONG
18     };
19
20     struct Animation {
21         QString name;
22         QList<QPixmap> frames;
23         float frameDuration;
24         PlayMode playMode;
25         bool isLooping;
26     };
27
28     explicit SpriteAnimation(QObject* parent = nullptr);
29     ~SpriteAnimation() override;
30
31     bool initialize();
32
33     // 动画管理
34     bool addAnimation(const QString& name, const QString& spriteSheetPath,
35                     int frameWidth, int frameHeight, int frameCount,
36                     float frameDuration = 0.1f, PlayMode mode = PLAY_LOOP);
37     bool hasAnimation(const QString& name) const;
38
39     // 播放控制
40     void play(const QString& animationName);
41     void stop();
42     void pause();
43     void resume();
44     bool isPlaying() const;
45     bool isPaused() const;
46
47     // 当前状态
48     QString getCurrentAnimation() const;
49     int getCurrentFrame() const;
50     QPixmap getCurrentFramePixmap() const;
51
52     // 播放速度
53     void setPlaySpeed(float speed);
54     float getPlaySpeed() const;
55
56     // 更新
57     void update(float deltaTime);
58
59     signals:
60     void animationStarted(const QString& name);
61     void animationFinished(const QString& name);
62     void frameChanged(int frame);

```

```

63
64 private:
65     QMap<QString, Animation> m_animations;
66     QString m_currentAnimation;
67     int m_currentFrame;
68     float m_frameTimer;
69     float m_playSpeed;
70     bool m_isPlaying;
71     bool m_isPaused;
72     bool m_isReverse;
73
74     void nextFrame();
75     void prevFrame();
76     bool loadSpriteSheet(const QString& path, int frameWidth, int frameHeight,
77                         int frameCount, QList<QPixmap>& frames);
78 };
79
80 #endif // SPRITEANIMATION_H

```

8.6 创建TileMap瓦片地图

```

tileMap.h
1  #ifndef TILEMAP_H
2  #define TILEMAP_H
3
4  #include <QObject>
5  #include <QVector>
6  #include <QPoint>
7  #include <QPixmap>
8  #include <QString>
9  #include <QMap>
10
11 class TileMap : public QObject
12 {
13     Q_OBJECT
14 public:
15     struct Tile {
16         int tileId;
17         bool isWalkable;
18         bool isTransparent;
19         int layer;
20     };
21
22     struct TileSet {
23         QString name;

```



```

24         QPixmap texture;
25         int tileWidth;
26         int tileHeight;
27         int columns;
28         int rows;
29     };
30
31     explicit TileMap(QObject* parent = nullptr);
32     ~TileMap() override;
33
34     bool initialize();
35
36     // 地图加载
37     bool loadMap(const QString& mapPath);
38     bool loadTileSet(const QString& name, const QString& path,
39                     int tileWidth, int tileHeight);
40
41     // 地图信息
42     int getMapWidth() const;
43     int getMapHeight() const;
44     int getTileWidth() const;
45     int getTileHeight() const;
46     int getLayerCount() const;
47
48     // 瓦片访问
49     int getTileId(int x, int y, int layer = 0) const;
50     bool setTileId(int x, int y, int tileId, int layer = 0);
51     bool isWalkable(int x, int y) const;
52
53     // 坐标转换
54     QPoint worldToTile(const QPointF& worldPos) const;
55     QPointF tileToWorld(int tileX, int tileY) const;
56
57     // 碰撞检测
58     bool checkCollision(const QRectF& rect) const;
59     QPointF findNearestWalkable(const QPointF& position) const;
60
61     // 渲染
62     void render(QPainter* painter, const QRectF& viewport);
63
64 private:
65     QVector<QVector<QVector<int>>>> m_tileData; // [layer][y]
66     [x]
67     QMap<QString, TileSet> m_tileSets;
68     QMap<int, bool> m_walkableTiles;
69
69     int m_mapWidth;

```

```

70     int m_mapHeight;
71     int m_tileWidth;
72     int m_tileHeight;
73     int m_layerCount;
74
75     bool parseTiledMap(const QString& mapPath);
76     QPixmap getTilePixmap(int tileId, const QString& tileSetName) const;
77 };
78
79 #endif // TILEMAP_H

```

验收标准

- ☐ render模块CMakeLists.txt配置正确
- ☐ 5个类的.h和.cpp文件全部创建
- ☐ CameraSystem支持平滑跟随、缩放、边界限制、屏幕震动
- ☐ ParticleSystem有基础的粒子和发射器管理
- ☐ SpriteAnimation支持多种播放模式
- ☐ TileMap支持瓦片地图加载和碰撞检测
- ☐ 代码能正常编译通过

十、步骤9：创建UI模块（ui）



目标：创建UI管理器类，搭建UI状态栈管理框架

包含类：UIManager

9.1 创建ui模块的CMakeLists.txt

```

src/ui/CMakeLists.txt

1  # UI界面模块
2  add_library(ui STATIC
3      uiManager.h
4      uiManager.cpp
5  )
6
7  target_link_libraries(ui PUBLIC
8      Qt6::Core

```

```

9      Qt6::Gui
10     Qt6::Widgets
11     core
12 )
13
14 target_include_directories(ui PUBLIC
15     ${CMAKE_CURRENT_SOURCE_DIR}
16 )

```

9.2 创建UIManager UI管理器

```

uiManager.h

1  #ifndef UIMANAGER_H
2  #define UIMANAGER_H
3
4  #include <QObject>;
5  #include <QStack>;
6  #include <QMap>;
7  #include <QString>;
8  #include <QWidget>;
9
10 class UIManager : public QObject
11 {
12     Q_OBJECT
13 public:
14     enum UIState {
15         UI_NONE,
16         UI_MAIN_MENU,
17         UI_SETTINGS,
18         UI_GAME_HUD,
19         UI_INVENTORY,
20         UI_REQUEST,
21         UI_DIALOG,
22         UI_SKILL,
23         UI_MAP,
24         UI_PAUSE_MENU,
25         UI_GAME_OVER
26     };
27
28     explicit UIManager(QObject* parent = nullptr);
29     ~UIManager() override;
30
31     bool initialize();
32
33     // UI状态栈管理

```

```
34     void pushState(UIState state);
35     void popState();
36     void changeState(UIState state);
37     UIState getCurrentState() const;
38     bool isStateActive(UIState state) const;
39     int getStateStackSize() const;
40     void clearStateStack();
41
42     // UI组件管理
43     QWidget* getUIWidget(UIState state) const;
44     void showUI(UIState state);
45     void hideUI(UIState state);
46     bool isUIVisible(UIState state) const;
47
48     // HUD更新
49     void updateHUD();
50     void showMessage(const QString& message, float duration = 3.0f);
51
52     // 对话框
53     void showDialog(const QString& speaker, const QString& text);
54     void hideDialog();
55     bool isDialogVisible() const;
56
57     // 更新
58     void update(float deltaTime);
59
60     signals:
61         void stateChanged(UIState newState);
62         void statePushed(UIState state);
63         void statePopped(UIState state);
64
65     private:
66         QStack<UIState> m_stateStack;
67         QMap<UIState, QWidget*> m_uiWidgets;
68         QMap<UIState, bool> m_visibility;
69
70         QString m_currentMessage;
71         float m_messageTimer;
72         bool m_messageVisible;
73
74         QString m_dialogSpeaker;
75         QString m_dialogText;
76         bool m_dialogVisible;
77
78         void enterState(UIState state);
79         void exitState(UIState state);
80         void initializeUIWidgets();
```

```
81     };
82
83     #endif // UIMANAGER_H
```

验收标准

- ☐ ui模块CMakeLists.txt配置正确
- ☐ UIManager类的.h和.cpp文件创建
- ☐ UI状态栈管理功能完整
- ☐ 代码能正常编译通过

十一、步骤10：更新主CMakeLists.txt和main.cpp



目标：配置主CMakeLists.txt，链接所有模块，更新程序入口

10.1 更新主CMakeLists.txt

CMakeLists.txt 完整内容

```
1  cmake_minimum_required(VERSION 3.20)
2  project(StarRetrospectTraveler VERSION 0.1.0 LANGUAGES CXX)
3
4  # C++标准设置
5  set(CMAKE_CXX_STANDARD 17)
6  set(CMAKE_CXX_STANDARD_REQUIRED ON)
7  set(CMAKE_CXX_EXTENSIONS OFF)
8
9  # 启用所有警告
10 if(MSVC)
11     add_compile_options(/W4)
12 else()
13     add_compile_options(-Wall -Wextra -Wpedantic)
14 endif()
15
16 # 编译选项配置
17 if(CMAKE_BUILD_TYPE STREQUAL "Debug")
18     add_compile_options(-g -O0)
19     if(NOT MSVC)
20         add_compile_options(-fsanitize=address -fno-omit-frame-pointer)
21         add_link_options(-fsanitize=address)
22     endif()
```

```
23 elseif(CMAKE_BUILD_TYPE STREQUAL "Release")
24     add_compile_options(-O3)
25 endif()
26
27 # MinGW-w64专用优化
28 if(MINGW)
29     add_compile_options(-static-libgcc -static-libstdc++)
30     add_link_options(-static-libgcc -static-libstdc++)
31     add_compile_options(-Wa,-mbig-obj)
32 endif()
33
34 # 查找Qt6模块
35 find_package(Qt6 REQUIRED COMPONENTS
36     Core
37     Gui
38     Widgets
39     Multimedia
40     Network
41     Svg
42     Xml
43 )
44
45 # 启用Qt的MOC、UIC、RCC自动处理
46 set(CMAKE_AUTOMOC ON)
47 set(CMAKE_AUTOUIC ON)
48 set(CMAKE_AUTORCC ON)
49
50 # 源文件目录（添加4个模块）
51 add_subdirectory(src/core)
52 add_subdirectory(src/game)
53 add_subdirectory(src/render)
54 add_subdirectory(src/ui)
55
56 # 主程序
57 add_executable(StarRetrospectTraveler
58     main.cpp
59 )
60
61 # 链接库（链接所有模块和Qt模块）
62 target_link_libraries(StarRetrospectTraveler PRIVATE
63     Qt6::Core
64     Qt6::Gui
65     Qt6::Widgets
66     Qt6::Multimedia
67     Qt6::Network
68     Qt6::Svg
69     Qt6::Xml
```

```

70     core
71     game
72     render
73     ui
74 )
75
76 # 包含目录
77 target_include_directories(StarRetrospectTraveler PRIVATE
78     ${PROJECT_SOURCE_DIR}/src
79 )
80
81 # 输出目录设置
82 set_target_properties(StarRetrospectTraveler PROPERTIES
83     RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin
84 )

```

10.2 更新main.cpp

main.cpp 完整内容

```

1  /**
2   * @file main.cpp
3   * @brief 《星溯旅人》游戏程序入口
4   * @author 星溯旅人开发团队
5   * @date 2026-06-21
6   */
7
8  #include <QApplication>;
9  #include <QDebug>;
10 #include "core/gameEngine.h"
11
12 int main(int argc, char* argv[])
13 {
14     QApplication app(argc, argv);
15
16     // 设置应用程序信息
17     QApplication::setApplicationName("StarRetrospectTraveler");
18     QApplication::setApplicationVersion("0.1.0");
19     QApplication::setOrganizationName("StarRetrospectStudio");
20
21     qDebug() << "《星溯旅人》游戏启动中...";
22     qDebug() << "版本: 0.1.0 (第一阶段 - 项目初始化)";
23
24     // 创建并初始化游戏引擎
25     GameEngine engine;
26     if (!engine.initialize()) {

```

```
27         qCritical() << "游戏引擎初始化失败!";
28         return -1;
29     }
30
31     // 启动游戏主循环
32     engine.start();
33
34     qDebug() << "游戏退出。";
35     return app.exec();
36 }
```

10.3 编译验证

1. 在Qt Creator中点击「构建」→「重新构建项目」
2. 检查编译输出，确保没有错误
3. 如果有警告，逐一检查并修复
4. 点击运行按钮，测试程序能否正常启动
5. 查看控制台输出，确认所有子系统初始化成功

验收标准

- ☐ 主CMakeLists.txt正确包含所有子模块
- ☐ main.cpp正确使用GameEngine
- ☐ 项目能完整编译通过，无错误
- ☐ 程序能正常运行，显示游戏窗口
- ☐ 控制台输出显示所有子系统初始化成功

十二、步骤11：创建配置文件和文档



目标：创建游戏配置文件模板、README文档、开发环境指南等配套文件

11.1 创建游戏配置JSON模板

文件：config/json/game_config.json

```
game_config.json
```

```
1  {
```



```
2  "version": "0.1.0",
3  "graphics": {
4      "window_width": 1280,
5      "window_height": 720,
6      "fullscreen": false,
7      "vsync": true,
8      "max_fps": 60,
9      "texture_quality": "high",
10     "particle_quality": "high"
11 },
12 "audio": {
13     "master_volume": 1.0,
14     "bgm_volume": 0.7,
15     "sound_volume": 0.8,
16     "voice_volume": 1.0,
17     "mute_on_focus_loss": true
18 },
19 "gameplay": {
20     "difficulty": "normal",
21     "auto_save": true,
22     "auto_save_interval": 300,
23     "language": "zh_CN",
24     "show_fps": false,
25     "show_minimap": true
26 },
27 "controls": {
28     "keyboard": {
29         "move_up": "W",
30         "move_down": "S",
31         "move_left": "A",
32         "move_right": "D",
33         "interact": "E",
34         "attack": "J",
35         "skill1": "K",
36         "skill2": "L",
37         "inventory": "I",
38         "quest": "Q",
39         "map": "M",
40         "pause": "Escape"
41     },
42     "mouse": {
43         "sensitivity": 1.0,
44         "invert_y": false
45     }
46 },
47 "video": {
48     "brightness": 1.0,
```

```
49         "contrast": 1.0,
50         "saturation": 1.0
51     }
52 }
```

11.2 创建Qt资源文件

文件: config/qrc/resources.qrc

```
resources.qrc
1  <!DOCTYPE RCC>;
2  <RCC version="1.0">;
3      <qresource prefix="/images">;
4          <!-- 图片资源将在此添加 -->;
5      </qresource>;
6      <qresource prefix="/audio">;
7          <!-- 音频资源将在此添加 -->;
8      </qresource>;
9      <qresource prefix="/fonts">;
10         <!-- 字体资源将在此添加 -->;
11     </qresource>;
12     <qresource prefix="/ui">;
13         <!-- UI素材将在此添加 -->;
14     </qresource>;
15     <qresource prefix="/config">;
16         <file>../../config/json/game_config.json</file>;
17     </qresource>;
18 </RCC>;
```

11.3 创建README.md

文件: README.md

```
README.md
1  # 星溯旅人 (StarRetrospectTraveler)
2
3  一款2.5D像素风格的银河恶魔城游戏，融合时间褶皱、星核共鸣、生态共生三大核心系统。
4
5  ## 项目信息
6
7  - **版本**: v0.1.0 (第一阶段 - 项目初始化)
8  - **引擎**: Qt 6.10.1
9  - **语言**: C++17
10 - **构建系统**: CMake 3.20+
```

```

11
12  ## 项目结构
13
14  ...
15  StarRetrospectTraveler/
16  |—— CMakeLists.txt      # CMake主配置文件
17  |—— main.cpp            # 程序入口
18  |—— README.md          # 项目说明
19  |
20  |—— src/                # 源代码目录
21  |   |—— core/           # 核心引擎模块 (8个类)
22  |   |—— game/          # 游戏逻辑模块 (10个类)
23  |   |—— render/        # 渲染系统模块 (5个类)
24  |   |—— ui/            # UI界面模块
25  |
26  |—— resources/          # 资源文件目录
27  |   |—— images/        # 图片资源
28  |   |—— audio/         # 音频资源
29  |   |—— fonts/         # 字体资源
30  |
31  |—— config/             # 配置文件目录
32  |   |—— json/          # JSON数据配置
33  |   |—— qrc/           # Qt资源文件
34  |
35  |—— docs/              # 文档目录
36  ...
37
38  ## 模块说明
39
40  ### Core (核心引擎)
41  - **GameEngine**：游戏引擎主类，管理主循环和各子系统
42  - **SceneManager**：场景管理器，负责场景切换和生命周期
43  - **ResourceManager**：资源管理器，负责资源加载和缓存
44  - **InputManager**：输入管理器，处理键盘、鼠标、手柄输入
45  - **AudioManager**：音频管理器，负责音乐和音效播放
46  - **GameConfig**：游戏配置，基于QSettings实现配置管理
47  - **SaveSystem**：存档系统，实现JSON存档
48  - **PluginManager**：插件管理器，负责插件加载和管理
49
50  ### Game (游戏逻辑)
51  - **Player**：玩家类，管理玩家属性和成长系统
52  - **Enemy**：敌人基类，各种敌人类型的父类
53  - **TimeSystem**：时间褶皱系统，管理时间线和时间操控
54  - **ResonanceSystem**：星核共鸣系统，管理共鸣等级和环境操控
55  - **EcosystemSystem**：生态共生系统，管理共生生物和生态平衡
56  - **CombatSystem**：战斗系统，管理战斗状态和伤害计算
57  - **SkillSystem**：技能系统，管理技能释放和冷却

```

```
58 - **AISystem**: AI系统, 敌人和共生生物的行为树
59 - **InventorySystem**: 背包系统, 管理物品和装备
60 - **QuestSystem**: 任务系统基础框架
61
62 ### Render (渲染系统)
63 - **RenderManager**: 渲染管理器
64 - **CameraSystem**: 相机系统, 实现跟随、缩放、震动效果
65 - **ParticleSystem**: 粒子效果系统, 实现各种视觉特效
66 - **SpriteAnimation**: 精灵动画组件, 管理帧动画播放
67 - **TileMap**: 瓦片地图系统, 实现关卡地图渲染
68
69 ### UI (界面)
70 - **UIManager**: UI管理器, 负责界面切换和生命周期
71
72 ## 开发环境搭建
73
74 ### 前置要求
75 - Qt 6.10.1 或更高版本
76 - CMake 3.20 或更高版本
77 - MinGW 64-bit 或 MSVC 2019+ 编译器
78 - Git 版本控制
79
80 ### 编译步骤
81
82 1. 克隆项目
83 ```bash
84 git clone <repository-url>
85 cd StarRetrospectTraveler
86 ```
87
88 2. 创建构建目录
89 ```bash
90 mkdir build
91 cd build
92 ```
93
94 3. 配置项目
95 ```bash
96 cmake .. -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH=/path/to/Qt6
97 ```
98
99 4. 编译项目
100 ```bash
101 cmake --build . -j$(nproc)
102 ```
103
104 5. 运行游戏
```

```
105  ```bash
106  ./bin/StarRetrospectTraveler
107  ```
108
109  ### 使用Qt Creator
110
111  1. 打开Qt Creator
112  2. 选择「打开文件或项目」
113  3. 选择项目根目录的 `CMakeLists.txt`
114  4. 配置构建套件
115  5. 点击运行按钮
116
117  ## 开发阶段
118
119  ### 第一阶段：项目初始化 
120  - [x] 项目结构搭建
121  - [x] CMake构建配置
122  - [x] 核心引擎框架（8个类）
123  - [x] 游戏逻辑框架（10个类）
124  - [x] 渲染系统框架（5个类）
125  - [x] UI系统框架（1个类）
126  - [x] Git版本控制
127  - [x] 基础配置文件
128
129  ### 第二阶段：核心引擎开发
130  - [ ] 模块系统
131  - [ ] 事件系统
132  - [ ] 资源管理完善
133  - [ ] 输入系统完善
134  - [ ] 场景系统完善
135  - [ ] 插件系统完善
136
137  ### 第三阶段：渲染系统开发
138  - [ ] 2.5D渲染实现
139  - [ ] 相机系统完善
140  - [ ] 精灵动画完善
141  - [ ] 瓦片地图完善
142  - [ ] 粒子系统完善
143
144  ### 第四阶段：游戏系统开发
145  - [ ] 玩家系统完善
146  - [ ] 敌人AI完善
147  - [ ] 三大核心系统实现
148  - [ ] 战斗系统实现
149  - [ ] 任务系统实现
150  - [ ] 背包系统实现
151
```

```
152  ## 开发规范
153
154  ### 代码规范
155  - 使用C++17标准
156  - 类名使用大驼峰命名法 (PascalCase)
157  - 函数名和变量名使用小驼峰命名法 (camelCase)
158  - 成员变量使用m_前缀
159  - 常量使用全大写加下划线
160  - 4空格缩进, 不使用Tab
161  - 每行不超过120字符
162
163  ### Git提交规范
164  ```
165  type(scope): description
166
167  类型说明:
168  - feat: 新功能
169  - fix: 修复
170  - docs: 文档
171  - style: 格式
172  - refactor: 重构
173  - test: 测试
174  - chore: 构建/工具
175  ```
176
177  ## 许可证
178
179  本项目仅供学习和研究使用。
180
181  ## 联系方式
182
183  - 项目仓库: [GitHub/Gitee链接]
184  - 问题反馈: [Issue链接]
185
```

11.4 创建开发环境配置文档

文件: docs/开发环境配置.md

开发环境配置.md

```
1  # 开发环境配置指南
2
3  ## Windows环境
4
5  ### 1. 安装Qt
```

```
6
7 1. 下载Qt Online Installer: https://www.qt.io/download
8 2. 运行安装程序, 注册/登录Qt账号
9 3. 选择「Custom」自定义安装
10 4. 选择Qt 6.10.1版本, 勾选以下组件:
11     - MinGW 64-bit
12     - Qt 5 Compatibility Module
13     - Qt Multimedia
14     - Qt Network
15     - Qt SVG
16     - Qt XML
17     - Qt Tools → Qt Creator
18     - Qt Tools → MinGW 64-bit Build Tools
19
20 ### 2. 安装CMake
21
22 - Qt Creator已自带CMake, 无需单独安装
23 - 如需独立安装: https://cmake.org/download/
24
25 ### 3. 安装Git
26
27 - 下载地址: https://git-scm.com/download/win
28 - 安装时选择「Git Bash Here」和「Git GUI Here」
29 - 配置用户信息:
30 ```bash
31 git config --global user.name "Your Name"
32 git config --global user.email "your.email@example.com"
33 ```
34
35 ### 4. 配置Qt Creator
36
37 1. 打开Qt Creator
38 2. 工具 → 选项 → 版本控制 → Git
39 3. 确认Git路径正确
40 4. 工具 → 选项 → 构建和运行 → 套件
41 5. 确认Desktop Qt 6.10.1 MinGW 64-bit套件可用
42
43 ## Linux环境
44
45 ### 1. 安装Qt
46
47 ```bash
48 # Ubuntu/Debian
49 sudo apt update
50 sudo apt install qt6-base-dev qt6-multimedia-dev qt6-svg-dev qt6-tools-dev
51 cmake build-essential git
52 ```
```

```
52
53   ### 2. 安装Qt Creator
54
55   ```bash
56   sudo apt install qtcreator
57   ```
58
59   ## macOS环境
60
61   ### 1. 安装Homebrew
62
63   ```bash
64   /bin/bash -c "$(curl -fsSL
65   https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
66   ```
67
68   ### 2. 安装Qt
69
70   ```bash
71   brew install qt cmake git
72   ```
73
74   ## 验证环境
75
76   ### 检查Qt版本
77
78   ```bash
79   qmake --version
80   # 或
81   qt6-cmake --version
82   ```
83
84   ### 检查CMake版本
85
86   ```bash
87   cmake --version
88   ```
89
90   ### 检查Git版本
91
92   ```bash
93   git --version
94   ```
95
96   ## 常见问题
97
98   ### Q: CMake找不到Qt
```



```
98 A: 设置CMAKE_PREFIX_PATH环境变量指向Qt安装路径:
99 ```bash
100 export CMAKE_PREFIX_PATH=/path/to/Qt/6.10.1/gcc_64
101 ```
102
103 ### Q: 编译时出现中文乱码
104 A: 在CMakeLists.txt中添加:
105 ```cmake
106 add_compile_options(-finput-charset=UTF-8 -fexec-charset=UTF-8)
107 ```
108
109 ### Q: MinGW编译太慢
110 A: 使用多线程编译:
111 ```bash
112 cmake --build . -j8
113 ```
114
```

验收标准

- ☐ game_config.json配置模板创建完成
- ☐ resources.qrc资源文件配置完成
- ☐ README.md项目说明文档创建完成
- ☐ 开发环境配置文档创建完成
- ☐ 所有文档内容完整、格式正确

十三、最终验证和总结



第一阶段完成标志：所有24个类创建完成，项目能正常编译运行，显示游戏窗口

最终检查清单

项目结构检查

- ☐ 14个目录全部创建
- ☐ 59个文件全部创建
- ☐ 4大模块划分清晰
- ☐ 资源、配置、文档目录齐全

代码质量检查

- ☐ 所有类都有头文件和实现文件
- ☐ 所有类都有initialize()方法
- ☐ 命名规范统一（大驼峰类名、小驼峰函数）
- ☐ 成员变量都有m_前缀
- ☐ 有完整的Doxygen注释
- ☐ 编译无错误
- ☐ 警告数量在合理范围内

功能验证

- ☐ 程序能正常启动
- ☐ 显示标题为"星溯旅人 - StarRetrospectTraveler"的窗口
- ☐ 窗口大小为1280x720
- ☐ 控制台输出所有子系统初始化成功
- ☐ 游戏主循环正常运行（每秒输出运行时间）
- ☐ 程序能正常退出

版本控制检查

- ☐ Git仓库已初始化
- ☐ .gitignore配置正确
- ☐ 所有代码已提交
- ☐ 提交信息规范

第一阶段完成统计

模块	类数量	包含类
core（核心引擎）	8个	GameEngine、SceneManager、ResourceManager、InputManager、AudioManager、GameConfig、SaveSystem、PluginManager
game（游戏逻辑）	10个	Player、Enemy、TimeSystem、ResonanceSystem、EcosystemSystem、CombatSystem、SkillSystem、AISystem、InventorySystem、QuestSystem

render（渲染系统）	5个	RenderManager、CameraSystem、ParticleSystem、SpriteAnimation、TileMap
ui（UI界面）	1个	UIManager
总计	24个	48个源文件（.h + .cpp）

核心架构特点

- **模块化设计：**4个模块各自作为独立静态库，便于维护和扩展
- **游戏主循环：**基于QTimer实现固定60FPS
- **场景系统：**基于QGraphicsView框架
- **输入系统：**支持按键映射，三态检测（pressed/held/released）
- **相机系统：**支持平滑跟随、缩放、边界限制、屏幕震动
- **UI状态栈：**支持多界面层级切换

下一步计划

1. 第二阶段：完善核心引擎各系统的具体实现
2. 第三阶段：实现渲染系统的完整功能
3. 第四阶段：实现游戏玩法系统和三大核心机制
4. 第五阶段：制作UI界面和游戏内容
5. 第六阶段：测试优化和发布



恭喜！ 第一阶段项目初始化已完成。项目版本：v0.1.0

（注：文档部分内容可能由 AI 生成）